

Joel Sivanish

Professor Kaur

Lab 2 – Introduction to Modern Symmetric Ciphers

1/23/2026

Introduction:

The purpose of this lab was to implement and analyze several fundamental building blocks used in modern symmetric cryptography. Unlike classical ciphers, modern block ciphers rely on low-level bitwise operations such as splitting data, swapping halves, rotating bits, and applying permutations. These operations form the internal structure of secure algorithms such as DES and AES.

In this lab, six cryptographic tasks were implemented in the C programming language. Each task represents a core transformation commonly used inside modern symmetric encryption algorithms. The goal of the lab was to understand how these operations manipulate data at the bit level and how they can be composed to form more complex cryptographic systems.

Task Descriptions:

Task 1: Split Word

- This task splits an n-bit word into two equal halves. The left half contains the most significant bits, while the right half contains the least significant bits. This operation is commonly used in Feistel-based cipher designs, where data is repeatedly divided and processed in halves.

Task 2: Combine Halves

- This task performs the inverse operation of Task 1 by combining two equal-sized halves into a single n-bit word. Care is taken to ensure that only the specified number of bits are included in the final result. This task demonstrates controlled bit composition and reinforces the importance of masking in cryptographic implementations.

Task 3: Swap Halves

- This task swaps the left and right halves of an n-bit word. Rather than implementing new bit manipulation logic, this operation reuses the split and combine functions from Tasks 1 and 2. This demonstrates modular design and mirrors the half-swapping step commonly found in block ciphers.

Task 4: Bit Rotation

- This task implements both left and right circular bit rotations. Rotation differs from simple shifting because bits that move past one end of the word wrap around to the opposite end. Bit rotations are frequently used in modern cryptographic algorithms to provide diffusion while remaining reversible.

Task 5: P-box Permutation

- This task implements a permutation box (P-box), which rearranges bits according to a permutation table. Bits are numbered from most significant to least significant, following the convention used in the lab instructions. The corrected example provided by the instructor was implemented and verified to ensure accuracy.

Task 6: Invert Permutation

- This task computes the inverse of a permutation table. Applying a permutation followed by its inverse restores the original ordering of bits. This task highlights the importance of reversibility in cryptographic design and demonstrates how permutation tables can be undone programmatically.

Implementation Details:

Each task was implemented in its own source and header file to maintain modularity and clarity. Shared helper functions, such as bit masking and binary printing, were placed in a utility file to avoid duplicated code.

A menu-driven interface was used to allow interactive execution of each task. For every task, the user can either:

1. Run the function with custom input values, or
2. Run predefined test cases that demonstrate examples from the lab PDF, including the corrected permutation example.

All input handling uses safe functions (fgets and strtol) to avoid input errors and undefined behavior.

Testing and Results:

Task 1 – Split Word

- Correctly separates input into left and right halves
- Handles varying bit sizes correctly

```
C:\Users\Joel\source\repos\C  X + v
=====
Task 1 Tests: Split
=====

--- Test Case 1 ---
Input:  240 (11110000)
Left:   15 (1111)
Right:  0 (0000)
Expected Left=15, Right=0

--- Test Case 2 ---
Input:  8004 (0001111101000100)
Left:   31 (00011111)
Right:  68 (01000100)
Expected Left=31, Right=68

--- Test Case 3 ---
Input:  255 (11111111)
Left:   15 (1111)
Right:  15 (1111)
Expected Left=15, Right=15

Press ENTER to continue...
```

Task 2 – Combine Halves

- Successfully reconstructs the original word from halves
- Properly masks results to the specified bit width

```
C:\Users\Joel\source\repos\C  X + v
=====
Task 2 Tests: Combine
=====

--- Test Case 1 ---
Left:   15 (1111)
Right:  0 (0000)
Output: 240 (11110000)
Expected Output=240 (11110000)

--- Test Case 2 ---
Left:   0 (0000)
Right:  15 (1111)
Output: 15 (00001111)
Expected Output=15 (00001111)

--- Test Case 3 ---
Left:   31 (00011111)
Right:  68 (01000100)
Output: 8004 (0001111101000100)
Expected Output=8004 (0001111101000100)

Press ENTER to continue...|
```

Task 3 – Swap Halves

- Correctly swaps halves using reusable components
- Produces expected output for known examples

```
C:\Users\Joel\source\repos\C X + v
=====
Task 3 Tests: Swap
=====

--- Test Case 1 ---
Input:  240 (11110000)
Output: 15 (00001111)
Expected Output=15 (00001111)

--- Test Case 2 ---
Input:  15 (00001111)
Output: 240 (11110000)
Expected Output=240 (11110000)

--- Test Case 3 ---
Input:  8004 (0001111101000100)
Output: 17439 (0100010000011111)
Expected Output=17439 (0100010000011111)

Press ENTER to continue...
```

Task 4 – Bit Rotation

- Correctly performs left and right circular rotations
- Handles rotation amounts larger than the bit width

```
C:\Users\Joel\source\repos\C X + v
=====
Task 4 Tests: Rotate
=====

--- Test Case 1 ---
Input:  12 (1100)
rotl2:  3 (0011)
Expected Output=3 (0011)

--- Test Case 2 ---
Input:  12 (1100)
rotr1:  6 (0110)
Expected Output=6 (0110)

--- Test Case 3 ---
Input:  129 (10000001)
rotl1:  3 (00000011)
Expected Output=3 (00000011)

Press ENTER to continue...|
```

Task 5 – P-box Permutation

- Correctly rearranges bits based on the permutation table
- Matches the corrected example provided by the instructor

```
C:\Users\Joel\source\repos\C  X + v
=====
Task 5 Tests: P-box Permute
=====

--- Test Case 1 ---
Input:  202 (11001010)
T = [5,6,1,4,3,7,8,2]
Output: 165 (10100101)
Expected Output=165 (10100101)

--- Test Case 2 ---
Input:  202 (11001010)
T = [1,2,3,4,5,6,7,8]
Output: 202 (11001010)
Expected Output=202 (11001010)

--- Test Case 3 ---
Input:  202 (11001010)
T = [8,7,6,5,4,3,2,1]
Output: 83 (01010011)
Expected Output=83 (01010011)
```

Task 6 – Invert Permutation

- Produces valid inverse permutation tables
- Applying a permutation followed by its inverse restores the original order

```
C:\Users\Joel\source\repos\C  X + v
=====
Task 6 Tests: Invert Permutation
=====

--- Test Case 1 ---
T = [6,3,4,5,2,1]
Inv = [6,5,2,3,4,1]
Expected Inv=[6,5,2,3,4,1]

--- Test Case 2 ---
T = [1,2,3,4,5,6]
Inv = [1,2,3,4,5,6]
Expected Inv=[1,2,3,4,5,6]

--- Test Case 3 ---
T = [2,3,4,5,6,1]
Inv = [6,1,2,3,4,5]
Expected Inv=[6,1,2,3,4,5]

Press ENTER to continue...
```

Conclusion:

In this lab, six core building blocks of modern symmetric cryptography were successfully implemented and tested. The exercise provided valuable insight into how real-world encryption algorithms manipulate data internally at the bit level. By focusing on correctness, clarity, and modular design, this lab bridges the gap between classical cryptography and modern cryptographic systems, laying the groundwork for understanding full block cipher implementations.